

last one-liner highlights SageMath's support for algebraic number fields through the function `QQbar()`, representing the field of all algebraic numbers. When evaluating `QQbar(sqrt(2))`, SageMath returns an exact algebraic object, but its displayed numerical representation might include a trailing question mark (?) to indicate that this displayed value is an approximation (1.414213562373095?). This ability to naturally handle symbolic numbers and exact algebraic objects is another fundamental difference setting SageMath apart from Python, where similar functionalities would necessitate importing multiple external packages like `SymPy`, `math`, or `mpmath`.

In addition to its extensive native functionalities, SageMath often delivers faster execution when performing mathematical operations involving large numbers. This performance advantage becomes especially relevant in the context of cryptography, where computations with very large integers are fundamental to public-key encryption schemes.

Let us now consider an example where the speed advantage of SageMath over Python is evident. The Python program named 'pythontime31.py' shown below measures the time taken to compute the product of two very large integers. These large integers, stored in variables *a* and *b*, are generated by using exponentiation operations.

```
import time
t0 = time.time()
a = 2**100000000
b = 3**100000000
c = a * b
print("Wall time:", time.time() - t0, "seconds")
```

The SageMath program named 'sagetime32.sage', shown below, performs the same operations as the Python program 'pythontime31.py'.

```
t0 = walltime()
a = 2^100000000
b = 3^100000000
c = a * b
print("Wall time:", walltime(t0), "seconds")
```

Notice that the two programs also illustrate the syntactical differences between Python and SageMath. For example, the programming languages use different operators for performing exponentiation (`**` in Python and `^` in SageMath). Upon executing the programs 'pythontime31.py' and 'sagetime32.sage', the outputs shown in Figure 4 are obtained. The outputs clearly show that the SageMath program runs roughly 16 times faster than the Python program, and as the sizes of numbers *a* and *b* increase, SageMath's performance advantage becomes even greater.

```
# python3 pythontime31.py
Wall time: 1.9711248874664307 seconds
# sage sagetime32.sage
Wall time: 0.12088227272033691 seconds
```

Figure 4: Python and SageMath speed comparison

Here's an important question: Why is SageMath significantly faster than Python for numeric processing? One of the key reasons SageMath outperforms standard Python in many number-theoretic and algebraic computations is its integration with powerful low-level libraries like FLINT (Fast Library for Number Theory) and GMP (GNU Multiple Precision Arithmetic Library). FLINT, written in C, is optimised for speed and designed specifically for number theory applications such as polynomial arithmetic, integer factorisation, and matrix operations over the integers and finite fields (all of them very relevant operations in public-key cryptography). GMP, on the other hand, provides high-performance arithmetic for large integers, rational numbers, and floating-point numbers with arbitrary precision (again very relevant in PKC). SageMath seamlessly connects to these libraries under the hood, giving it a significant computational advantage. In contrast, standard Python relies on the built-in `int` type and libraries like `math` or `decimal`, which are not designed for high-performance number theory or cryptographic workloads. As a result, SageMath handles very large number computations and symbolic algebra much faster and more accurately than vanilla Python (the standard, unmodified version of Python).

RSA under the microscope

Now, it is time to return to our discussion on RSA. In the previous article, we explored the history and background of the RSA algorithm. We will now move on to its implementation details. But before that, let us briefly recap what we have covered so far. In RSA, each user generates a pair of keys: a public key, which is shared openly, and a private key, which is kept secret. The public key is derived from two large prime numbers, but these primes themselves are never revealed. Anyone can use the public key to encrypt a message, but only the intended recipient—who possesses the corresponding private key—can decrypt it. This elegant asymmetry is what makes RSA a cornerstone of modern cryptography.

Now comes the important question: How large are the prime numbers used in practical implementations of RSA? Typically, RSA uses prime numbers that are 1024 bits (approximately 309 decimal digits) to 2048 bits (approximately 617 decimal digits) in size. A crucial part of the RSA public key is generated by multiplying two such large prime numbers. For example, in RSA-1024, two 1024-bit prime numbers are multiplied to form a